
Cross-Domain Recommender System

Movies → Games

IRS Project Report

A systematic experimental study of cross-domain recommendation
using the Amazon Reviews 2023 dataset

Leveraging movie preferences to recommend video games

Table of Contents

1. Executive Summary

2. Introduction

- 2.1 Background and Problem Statement
- 2.2 Research Methodology and Experiment Design
- 2.3 Aim and Objectives

3. Data

- 3.1 Data Source
- 3.2 Dataset Variants
- 3.3 Data Splitting Protocol

4. Methodology

- 4.1 Evaluation Protocol
- 4.2 MF-BPR: Matrix Factorization
- 4.3 NCF: Neural Collaborative Filtering
- 4.4 LightGCN: Graph Convolution
- 4.5 CMF: Collective Matrix Factorization
- 4.6 EMCDR: Cross-Domain Embedding Mapping
- 4.7 PTUPCDR: Personalized Transfer
- 4.8 SBERT-CDR: Content-Based Matching
- 4.9 Co-occurrence Reranking

5. Experiments and Results

- 5.1 Lesson 1: Explicit vs Implicit Ranking
- 5.2 Lesson 2: Low Overlap Kills CDR
- 5.3 Lesson 3: Overlap Filtering Rescues CDR
- 5.4 Lesson 4: Source-Rich / Target-Sparse
- 5.5 Lesson 5: Catalog Sharpening
- 5.6 Lesson 6: Cold-Start Protocol
- 5.7 Lesson 7: Content-Aware CDR (SBERT)
- 5.8 Lesson 8: Co-occurrence Reranking
- 5.9 Hyperparameter Analysis

6. Discussion

- 6.1 Routing Decision Rule
- 6.2 Key Insights

7. System Design

- 7.1 High-Level Architecture
- 7.2 Recommendation Flow
- 7.3 Frontend Architecture
- 7.4 Refresh Architecture
- 7.5 Key Design Decisions

8. Conclusion and Future Work

9. References

1. Executive Summary

This project investigates cross-domain recommendation (CDR) for transferring user preferences from movies to video games using the Amazon Reviews 2023 dataset. The central research question is: can a user's movie watching history improve game recommendations, particularly for users with little or no game interaction history?

We conduct a systematic, lesson-by-lesson experimental study comparing 8 recommendation models across 3 families: single-domain collaborative filtering (MF-BPR, NCF, LightGCN), cross-domain transfer models (CMF, EMCDR, PTUPCDR), and content-based models (SBERT, SBERT-CDR). Each lesson isolates a single experimental variable — user overlap ratio, source domain richness, catalog filtering, cold-start conditions, and post-processing — to understand what drives cross-domain transfer effectiveness.

Key Findings

(1) CDR models require high user overlap (100% vs 5.8%) to outperform single-domain baselines. (2) At cold-start (zero game history), CDR models achieve 4× better Recall@10 than LightGCN. (3) A simple, training-free co-occurrence reranking layer improves every model by 5–49% on standard evaluation and rescues single-domain models at cold-start (+289% for LightGCN). (4) SBERT content embeddings win 11× over collaborative models on niche/unpopular items. These findings justify a routing rule: select the model based on the user's game history depth and item popularity.

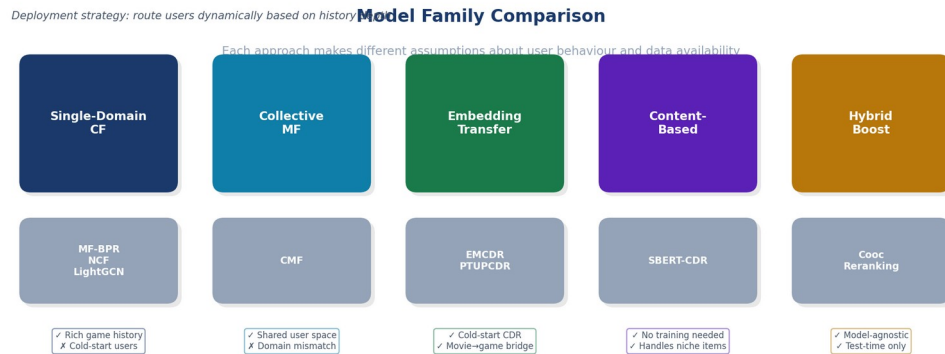


Figure 1: Model Family Strengths Across User Contexts

2. Introduction

2.1 Background and Problem Statement

Most recommendation systems are single-domain: they recommend items based solely on a user's interaction history within that domain. Cross-Domain Recommendation (CDR) transfers knowledge from a source domain (movies) to a target domain (games), addressing the cold-start problem — when a user has no target-domain history, CDR leverages their source-domain preferences.

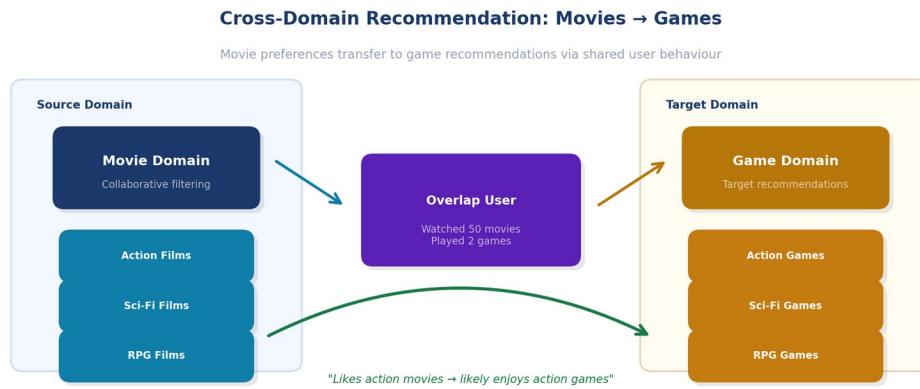


Figure 2: Cross-domain recommendation concept — transferring movie preferences to game recommendations

2.2 Research Methodology and Experiment Design

The experiments presented in this report are structured as a clean, progressive lesson series — but the path to this structure was far less linear. This section describes our actual research process, the challenges encountered, and how the final experiment design emerged from extensive exploratory work.

Phase 1: Literature Review and Infrastructure Setup

We began by studying the internal mechanisms of cross-domain recommendation algorithms — how models like EMCDD and PTUPCDD learn to map user preferences across domains, and how they differ from single-domain approaches like LightGCN that rely purely on within-domain interaction graphs. This informed our model selection: three CDR architectures (joint factorization, global mapping, personalized mapping) and three single-domain baselines (matrix factorization, neural, graph-based) to ensure fair comparison across model families.

We then built the base infrastructure: a data processing pipeline (raw Amazon reviews to filtered Parquet files), a standardized evaluation framework (full-rank and sampled protocols at $K=10$), and a benchmarking harness that ensures all models are evaluated on identical data splits. All models were implemented using publicly available libraries (RecBole, RecBole-CDR, PyTorch, scikit-learn) to avoid re-inventing the wheel and ensure reproducibility.

Phase 2: Exploratory Benchmarking and the CDR Disappointment

With the infrastructure in place, we ran extensive benchmarks comparing single-domain and cross-domain models across different data configurations. The initial results were consistently discouraging for CDR: LightGCN (a single-domain graph model) outperformed all CDR models in nearly every setting — including on user cohorts with rich movie history (>10 ratings) but sparse game history (3–5 ratings), precisely the regime where CDR is expected to excel.

We spent considerable effort trying different data configurations, user cohort filters, and hyperparameter settings. The CDR models consistently failed to demonstrate a clear advantage over the single-domain baseline. This raised a fundamental question: is the movie-to-game transfer signal in the Amazon dataset simply too weak for CDR to exploit?

Phase 3: Revisiting CDR Theory and Finding the Right Data Regime

Rather than concluding prematurely that CDR is ineffective — a conclusion contradicted by a large body of published research — we returned to the literature. Published CDR papers are evaluated under specific data conditions that we had not yet replicated: high user overlap between domains, rich source-domain history, and sparse or absent target-domain history.

Key Insight

The general-population Amazon dataset has only ~5.8% user overlap between movies and games — far too low for CDR mapping functions to learn from. Once we filtered to 100% overlap users and, crucially, tested on cold-start users with zero game history, CDR models finally outperformed their single-domain counterparts by 4×. The algorithms were not broken — the data regime was wrong.

Phase 4: Discovering Complementary Approaches

Along the way, we made two additional discoveries. First, a simple content-aware approach (SBERT sentence embeddings) performed surprisingly well — matching LightGCN overall and dominating 9–11× on niche/unpopular items. This reinforced a recurring theme: sophisticated model architecture does not guarantee better recommendations. What matters most is whether the model has access to the right signal for the right user segment.

Second, we explored hybrid approaches to augment the base models. After experimenting with various score fusion strategies, we discovered that a simple reranking of game candidates based on their co-occurrence with the user's liked movies improved scores for every model universally. This cross-domain item co-occurrence reranking requires no training and works as a model-agnostic post-processing layer — an unexpectedly powerful result.

Phase 5: From Exploration to Structured Lessons

With all findings in hand, we designed the final lesson series to present our discoveries as a progressive, pedagogically clear sequence. Each lesson isolates one experimental variable and builds on the previous lesson's conclusion. This structure was designed retrospectively — the actual research process involved many more experiments, several of which produced null results and were not included.

For example, in Lesson 5 (Catalog Sharpening), we hypothesized that removing long-tail movie items with very few ratings would produce denser, higher-quality source embeddings and improve CDR transfer. We tested genre whitelisting, overlap-user item filtering, and movie popularity thresholds as ablations. The result was largely null — filtering hurt single-domain LightGCN (smaller training catalog) and produced mixed results for CDR models. We included this lesson precisely because the null result is informative: it demonstrates that catalog manipulation is not a reliable lever for improving CDR, shifting attention to user-side factors (overlap ratio, source richness) instead.

In summary, our research methodology was deliberately exploratory: we ran a broad set of experiments, identified which factors genuinely drive CDR effectiveness (and which do not), and then distilled the findings into a curated lesson series that progresses from simple to complex. The lessons presented are the experiments that deepened our understanding — not necessarily the first experiments we ran.

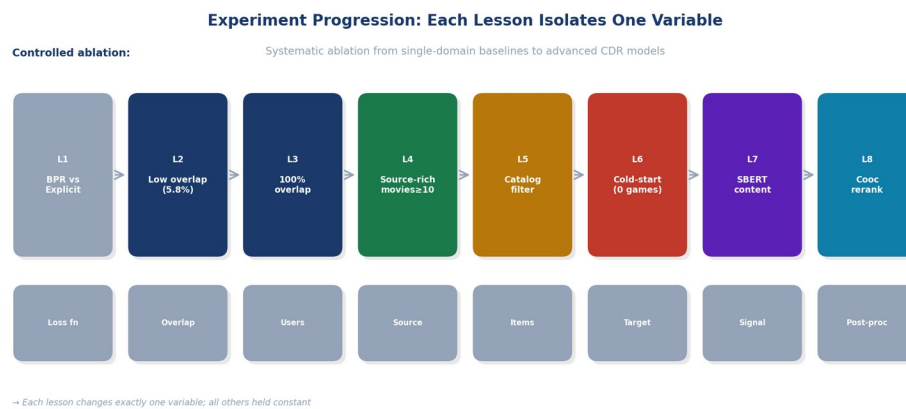


Figure 3: From Many Exploratory Experiments to 8 Curated Lessons

2.3 Aim and Objectives

- Compare explicit rating prediction vs implicit ranking approaches for recommendation (Lesson 1)
- Evaluate whether CDR models improve over single-domain models on naturally mixed populations with low user overlap (Lesson 2)
- Measure the impact of user overlap ratio on CDR effectiveness by filtering to 100% overlap users (Lesson 3)
- Investigate how source-domain richness (number of movie ratings) affects CDR transfer quality (Lesson 4)
- Test whether catalog sharpening (removing noisy items) improves CDR performance (Lesson 5)
- Evaluate CDR under true cold-start conditions where users have zero target-domain history (Lesson 6)
- Assess content-based SBERT embeddings as a cross-domain bridge for niche items (Lesson 7)
- Test co-occurrence reranking as a universal post-processing layer across all models and regimes (Lesson 8)

3. Data

3.1 Data Source

We use the Amazon Reviews 2023 dataset, a large-scale benchmark containing user reviews and ratings across multiple product categories. We focus on two domains: Movies and TV (user ratings of movies, DVDs, and streaming content) and Video Games (user ratings of video games across all platforms). Ratings are on a 1–5 star scale. We convert to implicit feedback by treating ratings ≥ 4 as positive interactions (POSITIVE_THRESHOLD = 4). This threshold is consistent across all experiments.

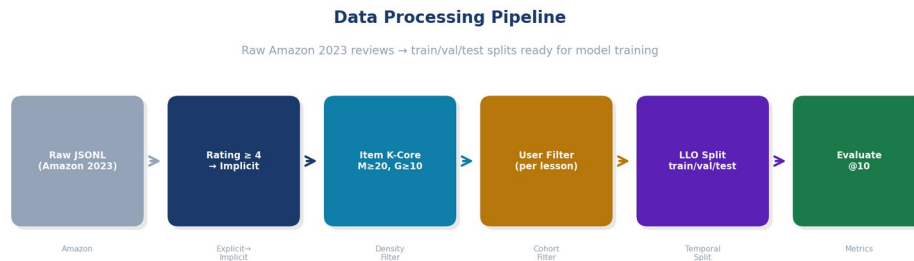


Figure 4: Data Processing Pipeline

3.2 Dataset Variants

Different lessons use different dataset configurations to isolate experimental variables:

Variant	Filter	Users	Movie Ints.	Game Ints.	Overlap %	Used in
Raw Mix	1,000,000	45,933	11,706	5.2%	L1–L2	No k-core
Overlap	19,880	40,288	10,034	100%	L3	movies $\geq$ 5, games $\geq$ 1
Src-Rich	14,328	39,534	9,147	100%	L4,L7	movies $\geq$ 10, games $\geq$ 1
Sharp	14,328	10,311	9,147	99.8%	L5	movie pop $\geq$ 10
Cold Start	12,654	40,288	10,034	100%	L6	User split 80/20

3.3 Data Splitting Protocol

We use leave-last-out (LLO) splitting on the target domain (games): for each user, the last game interaction by timestamp becomes the test item, the second-to-last becomes the validation item, and all earlier interactions from the training set. This simulates a realistic next-item prediction scenario. For Lesson 6 (cold-start), we use a user-split protocol: 80% of users train with their full game history, while 20% of users have their game

history withheld entirely, testing whether their movie history alone can generate useful game recommendations.

4. Methodology

This section explains every model we evaluate, starting from the simplest and building up to the most sophisticated. We assume no prior knowledge of recommendation systems — each model is introduced with an everyday analogy, then its mechanism and math are explained step by step. We group models into three families: single-domain models (trained on game data only), cross-domain models (trained on both movie and game data), and content-based models (using text descriptions rather than ratings).

4.1 Evaluation Protocol

Before explaining the models, we describe how we measure their quality. Imagine a teacher giving a student an exam with one correct answer: Evaluation protocol defines how hard that exam is.

We use two complementary protocols. In full-rank evaluation, the model must pick the correct answer from the entire catalog of ~9,000 items — like finding a specific book in a large library. In sampled evaluation, the model picks from just 100 items (1 correct + 99 random distractors) — like a multiple-choice exam. Both protocols report metrics at K=10.

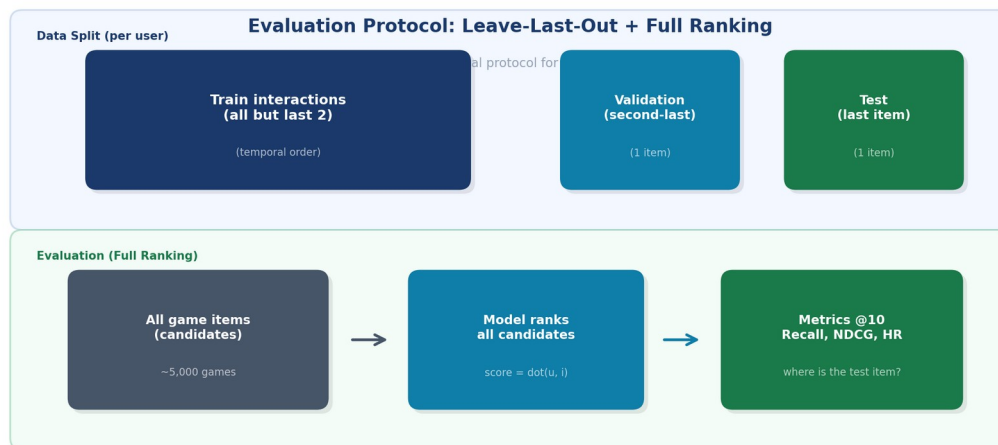


Figure 5: Full-Rank vs Sampled Evaluation Protocol

Recall@10 asks: did the correct item appear anywhere in the top 10? NDCG@10 goes further: was it ranked first (best), third (okay), or tenth (barely made it)? Higher positions earn more credit. Sampled metrics (HR@10, NDCG@10) are systematically higher than full-rank metrics because the task is much easier — choosing from 100 items vs 9,000+.

Single-Domain Models

The following three models are trained using game rating data only. They have no access to movie preferences.

Think of them as specialists: they only know about games and recommend games based on what similar gamers have played.

4.2 MF-BPR: Matrix Factorization with Bayesian Personalized Ranking

Analogy: Imagine you have a giant spreadsheet where rows are users and columns are games. Most cells are empty (users haven't played most games). MF-BPR tries to fill in the missing cells by compressing each user and each game into a short list of numbers (a "vector"). If a user's vector is similar to a game's vector, the model predicts the user will like that game.

Training. Training begins by assigning every user and every item a random vector of 64 numbers (drawn from a small Gaussian distribution). These vectors are the model's only learnable parameters. The model then iterates over 60 epochs, where each epoch samples millions of training triplets. A triplet consists of (user, positive item, negative item): the positive item is one the user actually interacted with (rated ≥ 4 stars), and the negative item is randomly sampled from items the user has never rated. For each triplet, the model computes two dot-product scores — one between the user vector and the positive item vector, and one between the user vector and the negative item vector. The BPR loss function then pushes the positive score to be higher than the negative score.

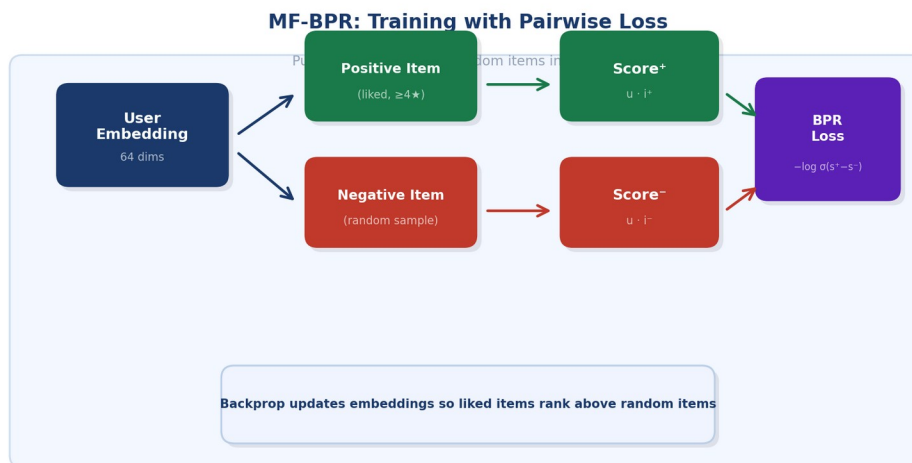


Figure 6: MF-BPR training process — user and item vectors learn from triplet sampling with BPR loss

BPR Loss. The Bayesian Personalized Ranking loss is elegant in its simplicity. For a user u , a positive item i^+ that the user liked, and a negative item i^- sampled at random, the loss is:

BPR Loss Function

$$\mathcal{L}_{\text{BPR}} = -\log \sigma(\mathbf{u}^\top \mathbf{i}^+ - \mathbf{u}^\top \mathbf{i}^-) + \lambda(\|\mathbf{u}\|^2 + \|\mathbf{i}^+\|^2 + \|\mathbf{i}^-\|^2)$$

$\sigma = \text{sigmoid}$; $\mathbf{u}^\top \mathbf{i}^+ = \text{user-positive score}$; $\mathbf{u}^\top \mathbf{i}^- = \text{user-negative score}$; $\lambda = L2 \text{ regularisation weight}$

Where σ is the sigmoid function that squashes the score difference into $[0,1]$. When the positive score is much higher than the negative score, the sigmoid output approaches 1 and the loss approaches 0. When the scores are close or inverted, the loss is large, and gradient descent updates the vectors to increase the gap. The regularization term λ prevents vectors from growing too large. Optimization uses plain SGD with learning rate 0.05 (high because there is no momentum or adaptive scaling).

Inference. After training completes, the model artifacts are the learned user embedding matrix U (shape: $n_users \times 64$) and item embedding matrix V (shape: $n_items \times 64$), saved as .npz files. To generate recommendations for a user u , we compute the dot product of u 's row in U with every row in V , producing a score for each item. Items are ranked by score in descending order, and the top-K items (excluding already-rated items) become the recommendations. This is a single matrix-vector multiplication, making inference extremely fast.

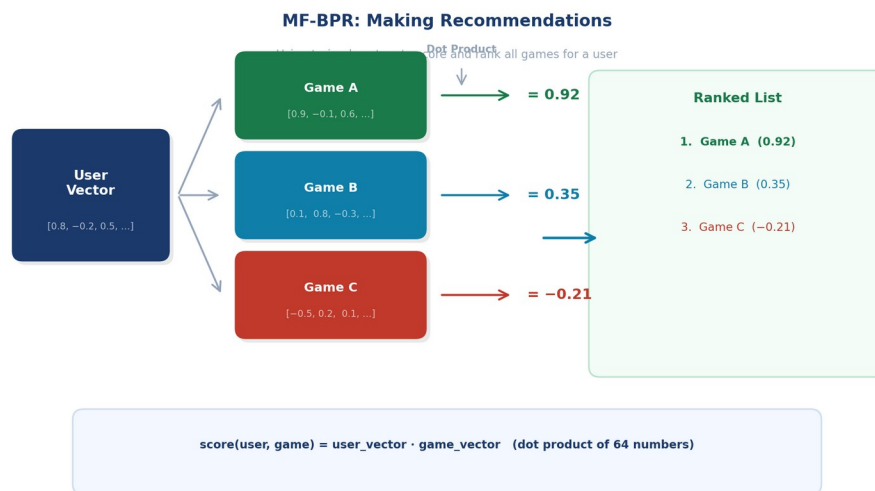


Figure 7: MF-BPR inference — dot product between user and item vectors produces ranking scores

Strengths: Simple, fast to train (~2 minutes), well-understood, and provides a strong baseline. **Weaknesses:** Each user and item is treated independently — the model can't leverage the fact that "users who like Game A also tend to like Game B" unless it discovers this pattern through the vectors alone. It also cannot handle cold-start users (no vector exists).

Parameter	Value	Notes
Embedding dimension	64	Each user/item = 64 numbers
Learning rate	0.05	High because plain SGD (no Adam)
Training epochs	60	Converges quickly on sparse data
Optimizer	NumPy SGD	Manual gradient descent
Regularization (λ)	0.01	L2 penalty on embedding norms

4.3 NCF (NeuMF): Neural Collaborative Filtering

Analogy: MF-BPR uses a simple multiplication to match users with items. NCF asks: what if we also pass the user and item vectors through a neural network, which can learn more complex patterns? It's like upgrading from a calculator to a computer — more flexible but also more complex.

Architecture. NCF has two parallel branches that process user-item pairs differently, then merges their outputs for a final prediction. Each branch maintains its own separate set of user and item embeddings (64 dimensions each), so the model has 4 embedding tables total.

The GMF (Generalized Matrix Factorization) branch performs element-wise multiplication of user and item vectors. Given user embedding u_{gmf} and item embedding i_{gmf} (both 64-dim), the output is $u_{\text{gmf}} \odot i_{\text{gmf}}$ — a 64-dimensional vector where each element is the product of the corresponding user and item values. This captures linear feature interactions, similar to standard matrix factorization.

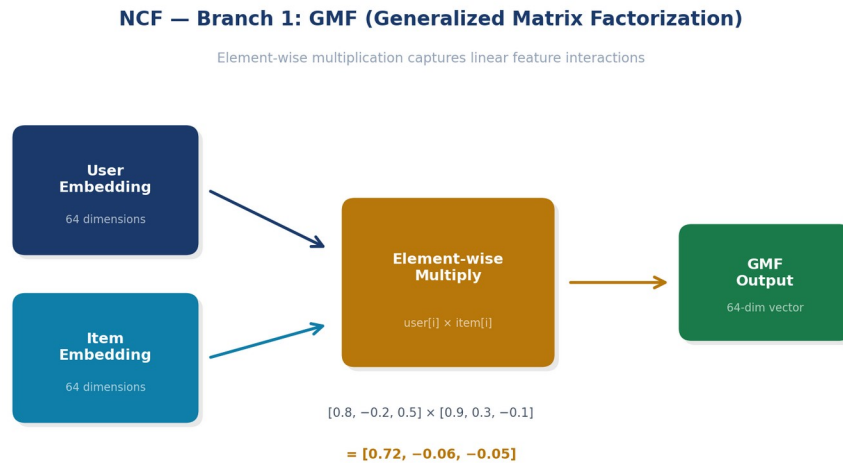


Figure 8: NCF — GMF branch: element-wise multiplication captures linear feature interactions

The MLP (Multi-Layer Perceptron) branch takes a different approach. It concatenates user and item embeddings into a single 128-dimensional input vector, then passes it through three hidden layers (128 → 64 → 32 neurons) with ReLU activations. Each layer applies a linear transformation followed by a non-linearity: $\text{output} = \text{ReLU}(W \cdot \text{input} + b)$. This allows the MLP to learn complex, non-linear patterns in user-item interactions that simple dot products cannot capture.

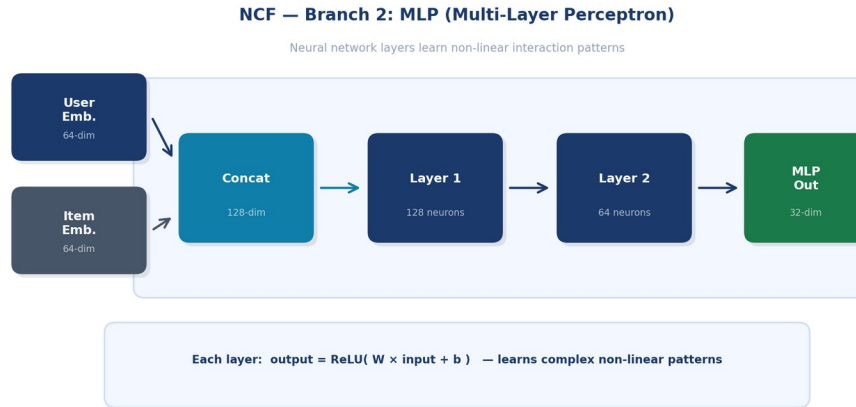


Figure 9: NCF — MLP branch: neural network layers learn non-linear user-item patterns

Combined Architecture. The final NeuMF (Neural Matrix Factorization) model concatenates the GMF output (64-dim) and MLP output (32-dim) into a 96-dimensional vector, then passes it through a single output layer with sigmoid activation to produce a score between 0 and 1.

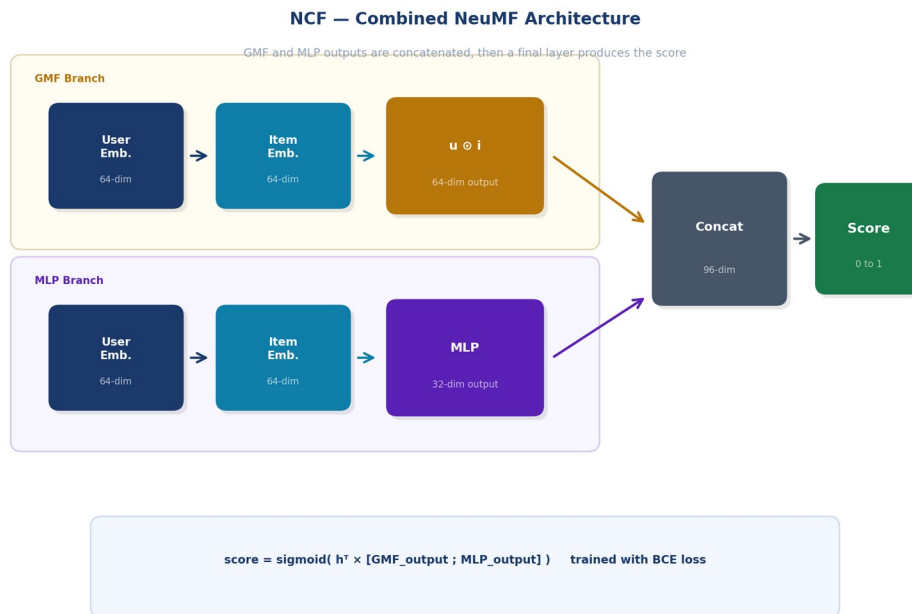


Figure 10: NCF — combined NeuMF architecture merging both branches into a final score

Training. NCF trains using Binary Cross-Entropy (BCE) loss on implicit feedback. Positive examples are user-item pairs where the user interacted with the item; negative examples are randomly sampled non-interactions (4 negatives per positive). For each batch of 256 user-item pairs, the model computes scores through both branches, merges them, and compares the predicted scores against the binary labels (1 for interaction, 0 for no interaction). The Adam optimizer adjusts all parameters (embeddings + MLP weights) with learning rate 0.001 over 150 epochs.

NCF Loss Function (BCE)

$$\hat{y} = \sigma(\mathbf{h}^\top [\text{GMF}(u, i); \text{MLP}(u, i)])$$

$$\mathcal{L} = - \sum_{(u, i)} [y \log \hat{y} + (1 - y) \log (1 - \hat{y})]$$

$y = 1$ for observed interactions; $y = 0$ for sampled negatives. Adam optimiser, $lr = 0.001$, 150 epochs.

Where $y=1$ for observed interactions and $y=0$ for sampled negatives. The BCE loss penalizes confident wrong predictions heavily: if the model predicts 0.9 for a negative item, the loss contribution is $-\log(0.1) = 2.3$, a strong gradient signal. This drives the model to clearly separate positives from negatives.

Inference. The trained artifacts include 4 embedding matrices (GMF user/item, MLP user/item) and the neural network weights (3 MLP layers + output layer). At inference, for a given user, the model must run a forward pass for every candidate item — unlike MF-BPR which uses a single matrix multiply. This makes NCF slower at inference but more expressive. In practice, we batch all items and run a single forward pass through PyTorch.

Parameter	Value	Notes
Embedding dimension	64	Separate embeddings for GMF and MLP
MLP layers	[128, 64, 32]	Three hidden layers with decreasing width
Negative samples	4 per positive	Implicit feedback training
Training epochs	150	Longer training needed (was 50, improved +30%)
Optimizer	Adam ($lr=0.001$)	Adaptive learning rate per parameter

4.4 LightGCN: Graph Convolution for Recommendations

Analogy: Imagine a social network where users and games are connected by "played" links. If Alice played Game X and Game Y, and Bob also played Game X, then Bob is "one hop away" from Game Y through Alice. LightGCN uses this network structure: a user's taste is shaped not just by what they played, but by what their "neighbors" (users with similar taste) played, and even what neighbors-of-neighbors played.

Graph Propagation. Users and items form a bipartite graph where each interaction creates an edge. LightGCN initializes random 96-dimensional embeddings for every node, then propagates information through the graph in $K=3$ rounds ("hops"). In each hop, a node's embedding is updated to be the weighted average of its neighbors' embeddings, with weights normalized by the square root of both nodes' degrees to prevent popular items from dominating.

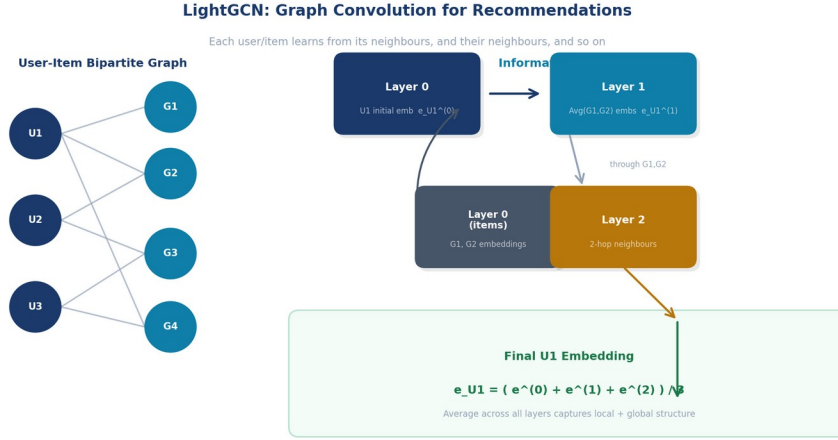


Figure 11: LightGCN graph propagation — information flows through multi-hop user-item connections

After K hops, each node has $K+1$ representations (from layer 0 to layer K). The final embedding is the simple mean of all layer representations. This layer-combination trick is critical: layer 0 captures the node itself, layer 1 captures direct neighbors, layer 2 captures 2-hop neighbors, and layer 3 captures 3-hop patterns. Averaging them balances local and global signals.

LightGCN Layer Propagation

$$\mathbf{e}_u^{(k)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|} \sqrt{|\mathcal{N}_i|}} \mathbf{e}_i^{(k-1)}$$

$$\mathbf{e}_u^* = \frac{1}{K+1} \sum_{k=0}^K \mathbf{e}_u^{(k)}$$

Symmetric sqrt-degree normalisation prevents popular items from dominating. $K=3$ hops, 96-dim embeddings.

The normalization factor $1/\sqrt{|\mathcal{N}_u|}\sqrt{|\mathcal{N}_i|}$ (symmetric sqrt-degree normalization) ensures that popular items with many connections don't overwhelm the signal from niche items. $K=3$ hops with 96-dimensional embeddings.

Training. LightGCN uses the same BPR loss as MF-BPR (positive items should score higher than random negatives), but the embeddings are first refined through graph propagation. In each training step: (1) sample a (user, positive, negative) triplet, (2) run $K=3$ layers of graph convolution to get refined embeddings, (3) compute dot-product scores, (4) compute BPR loss, (5) backpropagate gradients through all K layers back to the base embeddings. Only the base layer-0 embeddings are learnable; all propagation layers are parameter-free. This simplicity (no transformation matrices, no activation functions) is what makes LightGCN outperform more complex GCN variants.



Figure 12: LightGCN training loop — graph propagation followed by BPR loss optimization

Inference. After training, we pre-compute the final embeddings for all users and items by running the full K-layer propagation once. The resulting matrices ($n_{\text{users}} \times 96$ and $n_{\text{items}} \times 96$) are saved as .npz files. Inference is identical to MF-BPR: a single dot product between the user's final embedding and all item embeddings, then rank by score. LightGCN consistently achieves the best single-domain performance, outperforming MF-BPR by 70% and NCF by 140% on full-rank Recall@10.

Parameter	Value	Notes
Embedding dimension	96	Larger than MF/NCF for richer graph signal
GCN layers (K)	3	3-hop neighborhood propagation
Learning rate	0.001	Adam optimizer
Regularization	1e-4	L2 weight decay on base embeddings only
Batch size	2048	Large batches stabilize graph training

Cross-Domain Models

The following three models are trained using both movie and game data. They attempt to transfer knowledge from the movie domain to the game domain. Think of them as translators: they try to convert what they know about a user's movie taste into game recommendations. This is especially valuable for "cold-start" users who have watched movies but never played games.

4.5 CMF: Collective Matrix Factorization

Analogy: CMF is like a bilingual person who speaks both "Movie" and "Game." Instead of having separate user profiles for movies and games, CMF forces each user to have one single profile that must work for both domains simultaneously. If you like action movies, that same profile should also predict you like action games.

Training. CMF maintains three embedding matrices: a shared user matrix U ($n_{\text{users}} \times 96$), a movie item matrix V_{movie} ($n_{\text{movies}} \times 96$), and a game item matrix V_{game} ($n_{\text{games}} \times 96$). In each training epoch, the model alternates between movie and game batches. For a movie batch, it samples (user, positive_movie, negative_movie) triplets and computes BPR loss using U and V_{movie} . For a game batch, it samples (user, positive_game, negative_game) triplets and computes BPR loss using the same U and V_{game} . The crucial point is that the user embeddings U receive gradient updates from both domains, forcing them to encode preferences that work across movies and games.

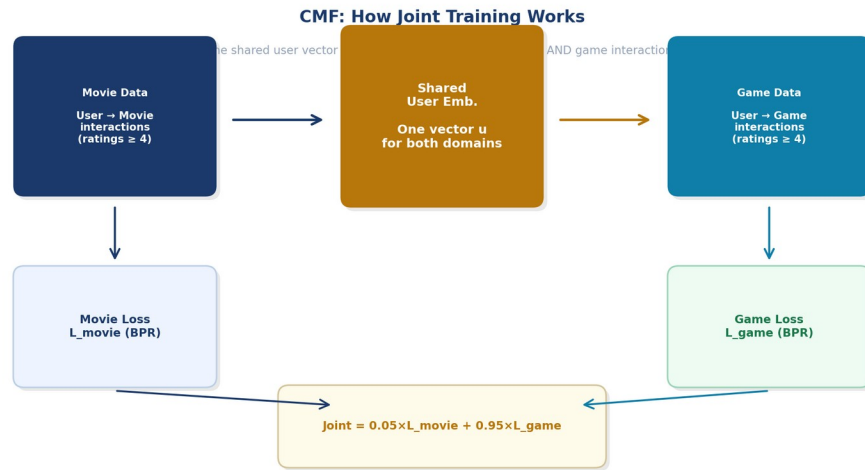


Figure 13: CMF mechanism — shared user embeddings trained jointly on movie and game interactions

CMF Joint Loss

$$\mathcal{L}_{\text{total}} = \alpha \mathcal{L}_{\text{BPR}}^{\text{movie}}(\mathbf{U}, \mathbf{V}_{\text{movie}}) + (1 - \alpha) \mathcal{L}_{\text{BPR}}^{\text{game}}(\mathbf{U}, \mathbf{V}_{\text{game}})$$

$\alpha = 0.05$ (5 % movie, 95 % game). Shared $\mathbf{U}$ receives gradients from both domains; $\mathbf{V}_{\text{movie}}$ and $\mathbf{V}_{\text{game}}$ update from their respective domains only.

Where $\alpha = 0.05$ heavily weights the game domain (95% game, 5% movie). Each $\mathcal{L}_{\text{BPR}}$ is the standard BPR loss over triplets in that domain. The shared user embedding $\mathbf{U}$ is updated by gradients from both losses, while $\mathbf{V}_{\text{movie}}$ and $\mathbf{V}_{\text{game}}$ only receive gradients from their respective domains. Adam optimizer with $\text{lr} = 0.0005$ over 60 epochs.

Inference. The trained artifacts are $\mathbf{U}$, $\mathbf{V}_{\text{movie}}$, and $\mathbf{V}_{\text{game}}$. For game recommendations, we compute score $= \mathbf{U}[\text{user}] \cdot \mathbf{V}_{\text{game}}^T$, identical to MF-BPR. The cross-domain transfer happens implicitly: because $\mathbf{U}$ was shaped by movie gradients too, it carries movie-derived preference signals into game scoring. For a cold-start game user who has movie history, their $\mathbf{U}$ vector already encodes useful information from movie training.

Limitations: CMF's "one profile fits all" approach is a blunt instrument. The shared embedding is a compromise that must be mediocre at both domains. In our experiments, CMF provides the weakest cross-domain transfer, especially at cold-start where the shared embedding has been shaped primarily by movie data (which constitutes 95% of interactions) and may encode movie-specific patterns that don't transfer to games.

Parameter	Value	Notes
-----------	-------	-------

Embedding dimension	96	Shared across both domains
Movie weight (α)	0.05	Focus on target domain (games)
Learning rate	0.0005	Adam optimizer
Training epochs	60	Joint training on both matrices
Batch alternation	Movie → Game	Alternates domain each batch

4.6 EMCDR: Cross-Domain Embedding Mapping

Analogy: Imagine you have a French-to-English dictionary. EMCDR works the same way, but for recommendation spaces. It first learns a "movie language" (movie embeddings) and a "game language" (game embeddings) separately, then trains a "translator" neural network that converts movie embeddings into game embeddings. For a cold-start user with only movie history, EMCDR translates their movie profile into game space to make game recommendations.

Training. EMCDR trains sequentially in three distinct phases, each producing specific artifacts. Phase 1 trains a complete MF-BPR model on movie interactions for 20 epochs, producing user embeddings U_{movie} and item embeddings V_{movie} (both 64-dim). Phase 2 independently trains a second MF-BPR model on game interactions for 20 epochs, producing U_{game} and V_{game} . Phase 3 is the key innovation: using only the overlap users (those who appear in both domains), it trains a mapping network f that transforms movie embeddings into game space: $f(U_{\text{movie}}[u]) \approx U_{\text{game}}[u]$.

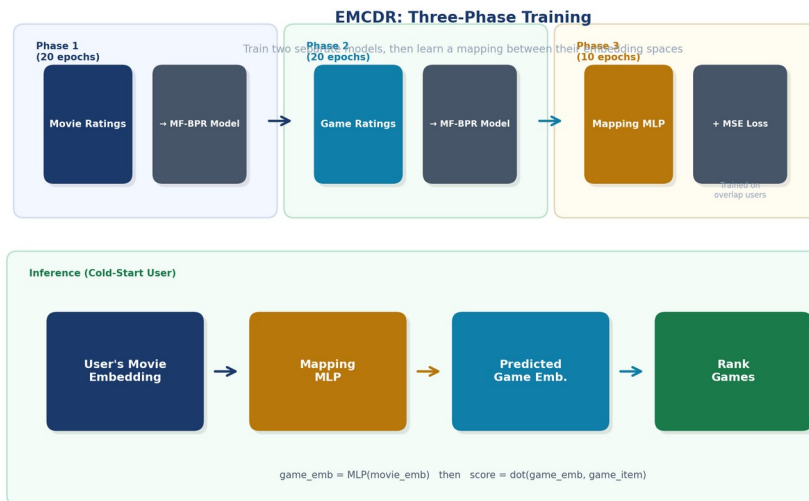


Figure 14: EMCDR three-phase training — separate domain models, then a learned mapping network

EMCDR – Three-Phase Training**Phase 1: Train movie MF-BPR** $\Rightarrow U_{\text{movie}}, V_{\text{movie}}$ **Phase 2: Train game MF-BPR** $\Rightarrow U_{\text{game}}, V_{\text{game}}$ **Phase 3 (mapping loss on overlap users $\mathcal{O}$):**

$$\mathcal{L}_{\text{map}} = \frac{1}{|\mathcal{O}|} \sum_{u \in \mathcal{O}} \|U_{\text{movie}}[u] - U_{\text{game}}[u]\|^2$$

MLP f: one hidden layer 64→64, ReLU activation, MSE loss, Adam optimiser.

Where $\mathcal{O}$ is the set of overlap users. The MLP has a single hidden layer (64→64) with ReLU activation, trained with Adam optimizer. The MSE loss pushes the mapped movie embedding to be as close as possible to the true game embedding for each overlap user.

Inference. The saved artifacts are V_{game} (game item embeddings) and the trained mapping network f . For any user — including cold-start users with zero game interactions — we first retrieve their movie embedding $U_{\text{movie}}[u]$ (from Phase 1), then compute $\text{mapped_emb} = f(U_{\text{movie}}[u])$. The game score for each item is then $\text{mapped_emb} \cdot V_{\text{game}}[\text{item}]$. This makes EMCDR especially powerful at cold-start (Lesson 6: 5× over LightGCN) because any user with movie history can be translated.

Parameter	Value	Notes
Embedding dimension	64	Both movie and game embeddings
Phase 1 epochs	20	Movie MF-BPR training
Phase 2 epochs	20	Game MF-BPR training
Phase 3 epochs	10	Mapping network training
Mapping MLP	64→64 (1 hidden layer)	ReLU activation, MSE loss
Requires overlap users	Yes	Only overlap users train the mapping

4.7 PTUPCDR: Personalized Transfer with Experts

Analogy: If EMCDR is one universal translator, PTUPCDR is a team of specialist translators plus a manager. There are 8 "expert" networks, each good at translating a different type of movie taste. For each user, a "gate" network decides which experts to listen to and how much weight to give each one. A sci-fi fan might rely heavily on Expert 3 (the sci-fi specialist), while a comedy fan might rely on Expert 7.

Training. Like EMCDR, PTUPCDR starts with Phase 1 (movie MF-BPR) and Phase 2 (game MF-BPR) to obtain domain-specific embeddings. The key innovation is in Phase 3: instead of a single mapping MLP, PTUPCDR uses a Mixture of Experts (MoE) architecture. The system has $K=8$ expert networks, each a small MLP (64→64). A gate network $g(u_{\text{movie}}) = \text{softmax}(W_{\text{gate}} \cdot u_{\text{movie}})$ takes the user's movie embedding and outputs 8 weights (one per expert). The mapped embedding is the weighted sum of all expert outputs.

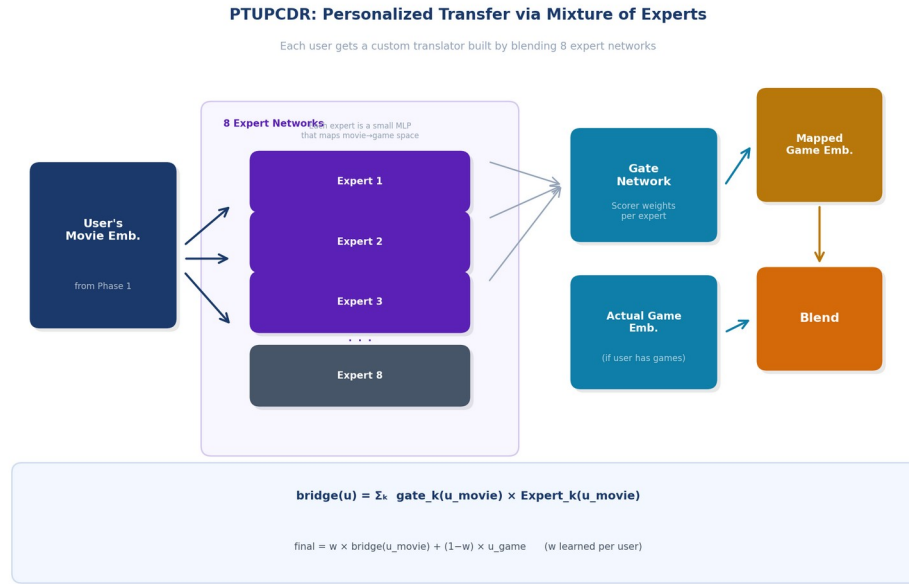


Figure 15: PTUPCDR — personalized transfer via mixture of 8 expert networks with gating

PTUPCDR – Bridge and Blend

$$\text{bridge}(u) = \sum_{k=1}^8 \text{gate}_k(u_{\text{movie}}) \cdot \text{Expert}_k(u_{\text{movie}})$$

$$\text{gate}(u_{\text{movie}}) = \text{softmax}(\mathbf{W}_{\text{gate}} u_{\text{movie}}) \Rightarrow 8 \text{ weights summing to } 1$$

$$\text{emb}_{\text{final}}(u) = w \cdot \text{bridge}(u_{\text{movie}}) + (1-w) \cdot u_{\text{game}}$$

$w = 1$ for cold-start users (no u_{game}). Minimises MSE between $\text{emb}_{\text{final}}$ and u_{game} over overlap users.

Where w is a learned scalar per user. For cold-start users (no u_{game}), $w=1$ and the model relies entirely on the bridge. For warm users, the blend balances translated movie signal with direct game signal. Training minimizes MSE between final_emb and u_{game} over overlap users, with Adam optimizer.

Inference. The saved artifacts include V_{game} , the 8 expert MLPs, the gate network weights, and the blend weight w . At inference, given a user's movie embedding: (1) the gate network produces 8 expert weights, (2) each expert maps the movie embedding, (3) outputs are weighted-summed to produce the bridge embedding, (4) if the user has game history, the bridge is blended with their game embedding, (5) the final embedding is dot-producted with V_{game} to score all games. PTUPCDR excels in the "few-shot" scenario (Lesson 4): when a user has 1-2 game interactions, the blend formula combines the translated movie profile with the sparse game signal.

Parameter	Value	Notes
Embedding dimension	64	Same as EMCDR for fair comparison
Number of experts (K)	8	8 specialist translator networks

Expert architecture	64→64 MLP	Each expert is a small network
Gate network	Linear + softmax	Produces per-expert weights
Blend weight (w)	Learned per user	Balances movie vs game signal

Content-Based and Post-Processing Models

The remaining two approaches don't rely on collaborative filtering. SBERT matches items by their text descriptions, while co-occurrence reranking boosts scores based on statistical patterns in the data. Both are training-free and model-agnostic.

4.8 SBERT-CDR: Content-Based Semantic Matching

Analogy: All the models above work by analyzing who rated what (collaborative filtering). SBERT takes a completely different approach: it reads the text description of each item and matches them by meaning. If a movie is described as "a thrilling sci-fi adventure with robots" and a game is described as "sci-fi action with robot enemies," SBERT knows they are semantically similar — even if no user has ever rated both.

Encoding. SBERT-CDR uses a pre-trained Sentence-BERT model (all-MiniLM-L6-v2, 22M parameters) to encode each item's text into a dense 384-dimensional vector. The input text is formatted as "title: [title]. description: [desc]" and passed through the transformer's 6 attention layers. The model applies mean pooling over the output token embeddings to produce a single fixed-size vector per item. Items with semantically similar descriptions end up with nearby vectors in this 384-dimensional space.

SBERT: Sentence Embedding Process

Text is tokenised, encoded via transformer, then mean-pooled to a fixed vector

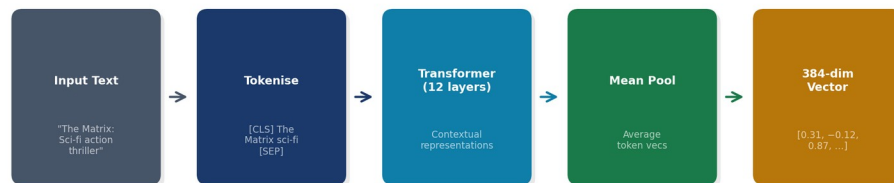


Figure 16: SBERT encoding — item text is transformed into a 384-dim semantic vector

Cross-Domain Mechanism. The key insight is that movies and games are encoded into the same 384-dimensional space using the same encoder. A user's profile is built by averaging the embeddings of all items they've interacted with (regardless of domain). Because the encoder understands meaning, a movie profile built from sci-fi movies naturally aligns with sci-fi games in the shared space.

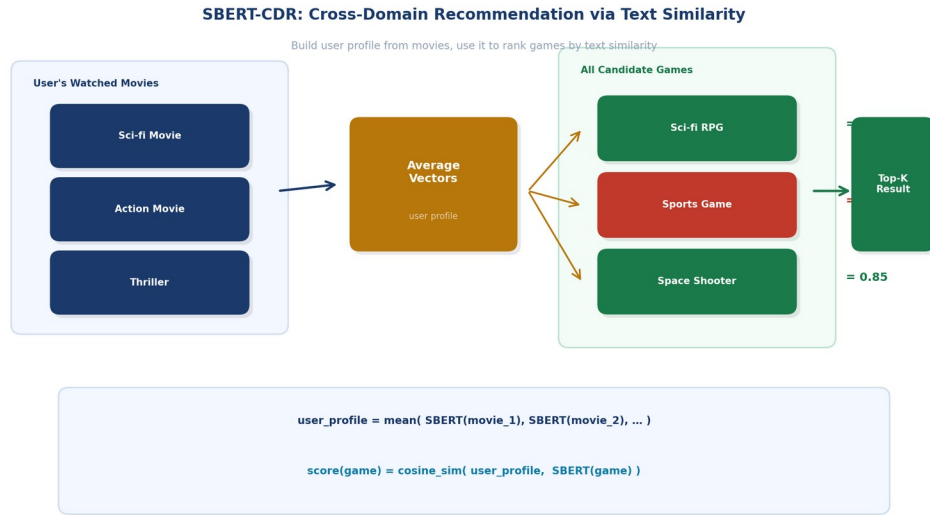


Figure 17: SBERT-CDR cross-domain flow — movie profiles match game items through shared semantic space

SBERT Scoring (No Training Required)

$$\mathbf{e}_i = \text{SBERT}(\text{"title": [t], desc": [d]}) \in \mathbb{R}^{384}$$

$$\mathbf{p}_u = \frac{1}{|\mathcal{H}_u|} \sum_{i \in \mathcal{H}_u} \mathbf{e}_i$$

$$\text{score}(u, g) = \cos(\mathbf{p}_u, \mathbf{e}_g) = \frac{\mathbf{p}_u \cdot \mathbf{e}_g}{\|\mathbf{p}_u\| \|\mathbf{e}_g\|}$$

$\mathcal{H}_u$ = user's interaction history. Pre-trained encoder used as-is — zero fine-tuning required.

No training or fine-tuning is performed. The pre-trained encoder is used as-is. This makes SBERT-CDR the only model that requires zero training data from the target domain.

Inference. The artifacts are simply the pre-computed item embedding matrix ($n_{\text{items}} \times 384$). At inference, we compute the user profile as the mean of their rated items' embeddings, then compute cosine similarity against all game embeddings. SBERT-CDR is the only model that can recommend niche/unpopular items effectively (Lesson 7: 9× over LightGCN on niche items), because it doesn't need popularity or rating volume — just good text descriptions. However, it can miss collaborative patterns ("users who liked X also liked Y") because it only considers text content.

Parameter	Value	Notes
Encoder model	all-MiniLM-L6-v2	22M params, 6 transformer layers
Embedding dimension	384	Fixed by the encoder architecture
Training required	None	Pre-trained; no fine-tuning needed
Cross-domain mechanism	Shared text space	Movies and games encoded identically
Similarity metric	Cosine similarity	Normalized dot product

4.9 Co-occurrence Reranking

Analogy: Imagine you're at a bookstore and a friend says "people who bought the same books you bought also bought THIS game." Co-occurrence reranking works exactly like this: it looks at users who watched the same movies as you and checks which games they also played. Games that frequently co-occur with your movies get a score boost.

Matrix Construction. For every (movie, game) pair in the training data, we count how many overlap users rated both items. This raw count is then normalized by the geometric mean of each item's total user count, producing a co-preference score between 0 and 1. The resulting matrix C has shape $(n_movies \times n_games)$, where $C[m][g]$ indicates how strongly movie m and game g are associated across the user population. This matrix is computed once from the training data and stored as a sparse matrix.

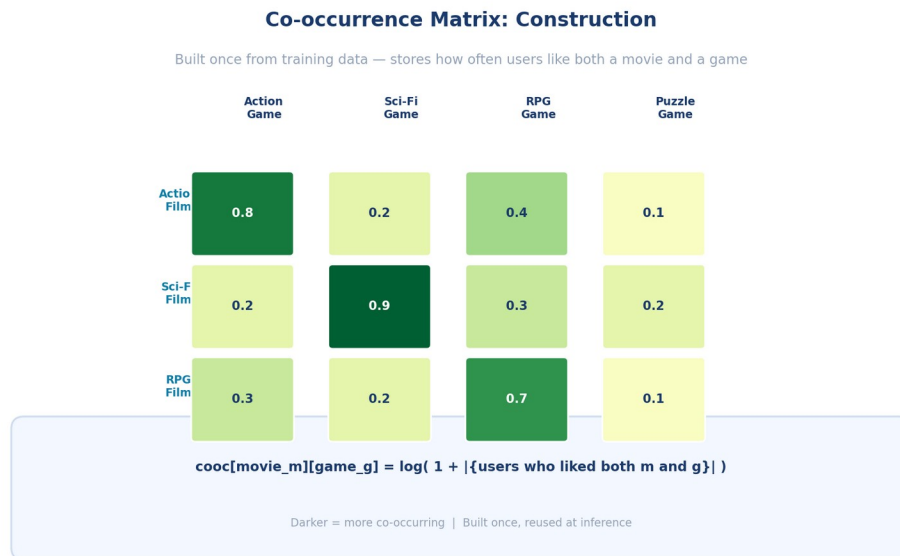


Figure 18: Co-occurrence matrix construction — counting and normalizing cross-domain user overlap

Inference. Co-occurrence reranking is applied as a post-processing layer on top of any base model's scores. For a given user, we first obtain the base model's score for each candidate game. Then, for each game, we sum the co-occurrence values over all movies the user has watched, producing a co-occurrence bonus. This bonus is scaled by a tunable strength parameter λ and added to the base score. The final ranking is by combined score.

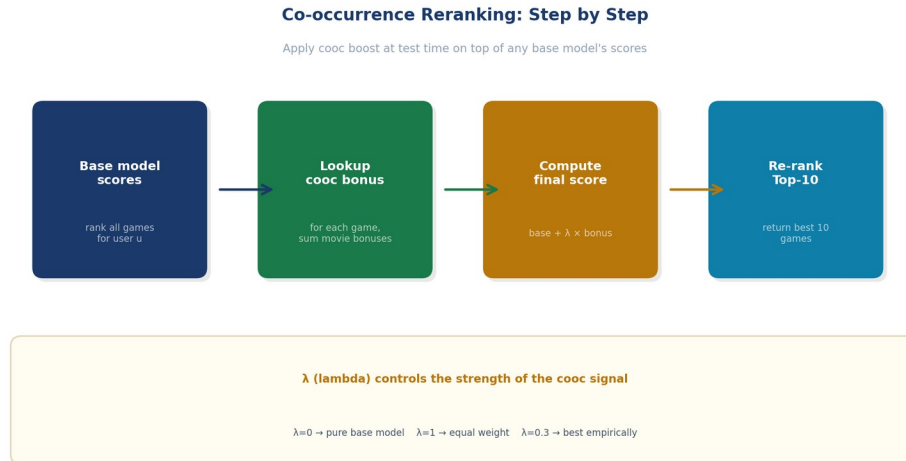


Figure 19: Co-occurrence reranking at inference — base model scores are boosted by cross-domain signals

Co-occurrence Scoring Formula

$$\text{cooc}(m, g) = \frac{\text{users who rated both}}{\sqrt{\text{rated } m \cdot \text{rated } g}}$$

$$\text{bonus}(u, g) = \sum_{m \in \mathcal{M}_u} \text{cooc}(m, g)$$

$$\text{score}_{\text{final}}(g) = \text{score}_{\text{base}}(g) + \lambda \cdot \text{bonus}(u, g)$$

$\mathcal{M}_u$ = user's rated movies; λ tuned per model via grid search. No training — computed directly from interaction counts.

The λ parameter is tuned per base model via grid search. No training is involved — the matrix is computed directly from interaction counts. This makes co-occurrence reranking deterministic, reproducible, and extremely fast to apply.

Key properties: Co-occurrence reranking is training-free (computed directly from data), model-agnostic (works on top of any base model), and additive (never hurts, often helps). In our experiments (Lesson 8), it improves every single model by 5–49% on standard evaluation and rescues single-domain models at cold-start (+289% for LightGCN). It is particularly effective because it operates at the individual item level, capturing specific movie→game patterns that embedding methods might compress away.

Property	Value	Notes
Training required	None	Computed from co-occurrence counts
Model-agnostic	Yes	Applied on top of any base model
Strength parameter (λ)	Tuned per model	Grid search over [0.1, 0.5, 1, 2, 5]
Cold-start boost	+289% (LightGCN)	Dramatic improvement for weak models
Matrix density	Sparse	Only (movie,game) pairs with shared users

5. Experiments and Results

Each lesson isolates a single experimental variable to build understanding incrementally. We report Recall@10 (full-rank) as the primary metric throughout.

5.1 Lesson 1: Explicit vs Implicit Ranking

Experiment Setup

Variable Changed: Loss function (BPR pairwise ranking vs RMSE regression). Held Constant: Model architecture (MF), data (1M users), evaluation protocol.

Expectation: BPR should outperform because it directly optimizes the ranking objective that evaluation metrics measure.

Model	Loss Function	Recall@10	NDCG@10
MF-BPR	BPR pairwise ranking	0.0130	0.0067
MF-Explicit	RMSE on star ratings	0.0025	0.0010

Key Finding

Confirmed — BPR wins 5.2× on Recall@10. The ranking loss directly optimizes the quantity that evaluation metrics measure, while RMSE optimizes prediction accuracy on ratings that may not correlate with top-K relevance.

Lesson 1 — Model Comparison

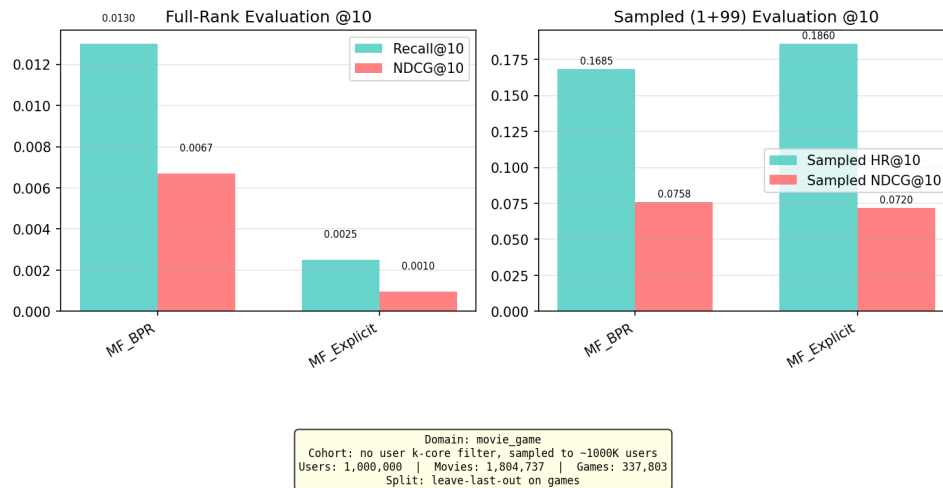


Figure 20: Lesson 1 — MF-BPR vs MF-Explicit comparison

5.2 Lesson 2: Low Overlap Kills CDR

Experiment Setup

Variable Changed: Model architecture — comparing 3 single-domain models (MF-BPR, NCF, LightGCN) vs 3 CDR models (CMF, EMCDR, PTUPCDR).

Held Constant: Data (1M users, 5.2% overlap), loss function (BPR for all), evaluation protocol (full-rank + sampled).

Expectation: CDR models should outperform single-domain baselines because they can leverage movie history. However, the very low overlap (5.2%) may starve the cross-domain mapping functions of training data.

Model	Family	Recall@10	NDCG@10	Gap vs LightGCN
LightGCN	Graph (single)	0.0290	0.0157	—
MF-BPR	MF (single)	0.0170	0.0093	-41%
EMCDR	Mapping (CDR)	0.0160	0.0082	-45%
NCF	Neural (single)	0.0120	0.0069	-59%
PTUPCDR	Personal mapping (CDR)	0.0085	0.0054	-71%
CMF	Joint MF (CDR)	0.0050	0.0022	-83%

Key Finding

All CDR models lose to LightGCN by 45–83% on full-rank Recall@10. With only 5.2% user overlap, the cross-domain mapping functions lack sufficient training data. Interestingly, CDR models score very high on sampled evaluation ($HR@10 > 0.80$), revealing a protocol gap: sampled metrics can be misleading.

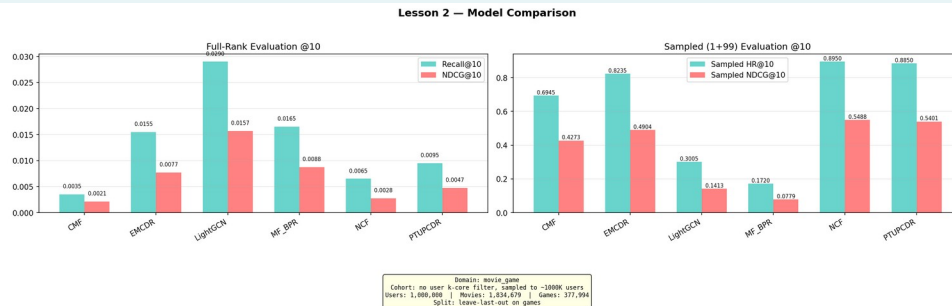


Figure 21: Lesson 2 — All models on mixed population (5.8% overlap)

5.3 Lesson 3: Overlap Filtering Rescues CDR

Experiment Setup

Variable Changed: User overlap — from 5.2% to 100% by filtering to users active in both domains.

Held Constant: Model set, evaluation protocol, loss function.

Expectation: With 100% overlap, CDR models should dramatically improve as mapping functions now have sufficient shared users to train on.

Model	Recall@10	NDCG@10
LightGCN	0.0595	0.0316
MF-BPR	0.0445	0.0224
CMF	0.0395	0.0220
PTUPCDR	0.0315	0.0176

NCF	0.0255	0.0144
EMCDR	0.0225	0.0139

Key Finding

All scores improve dramatically vs L2. CMF jumps from 0.005 to 0.040 (8×). PTUPCDR from 0.009 to 0.032 (3.5×). The 100% overlap dataset provides sufficient shared users for CDR mapping functions to learn meaningful cross-domain relationships.

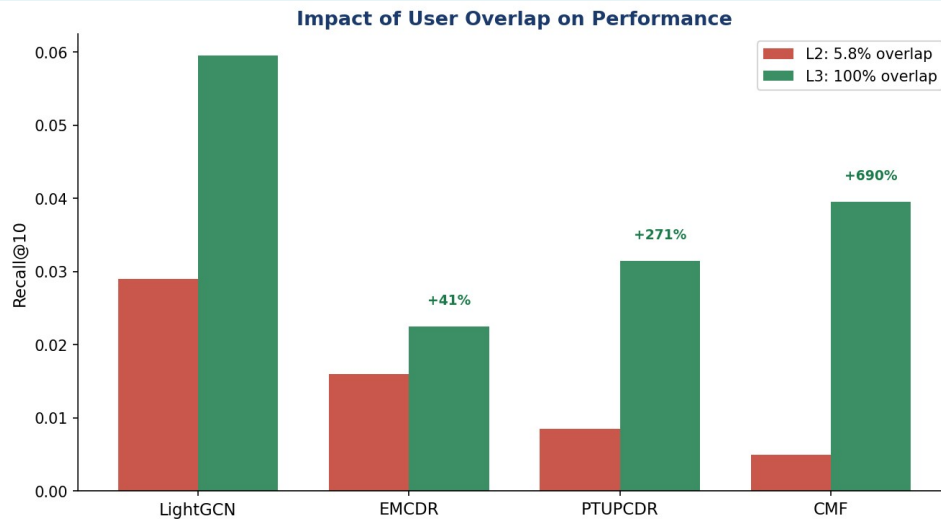


Figure 22: Impact of user overlap — CDR models gain 40–690% when overlap increases from 5.8% to 100%

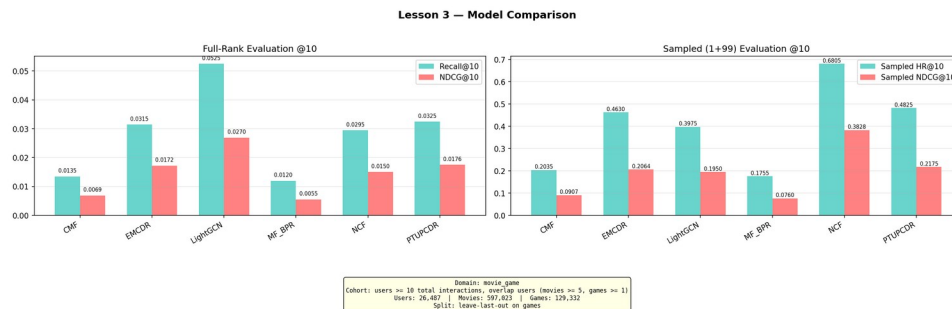


Figure 23: Lesson 3 — 100% overlap users (19,880 users)

5.4 Lesson 4: Source-Rich / Target-Sparse

Experiment Setup

Variable Changed: Source-domain richness — requiring ≥ 10 movie ratings (vs ≥ 5 in L3).

Held Constant: 100% overlap, evaluation protocol.

Expectation: Richer movie history should give CDR models better source embeddings to transfer from.

Model	Recall@10	Gap vs LightGCN
LightGCN	0.0380	—
PTUPCDR	0.0315	-17%

EMCDR	0.0265	-30%
NCF	0.0225	-41%
CMF	0.0140	-63%
MF-BPR	0.0095	-75%

Key Finding

PTUPCDR closes to within 17% of LightGCN (was 71% gap in L2). Richer source embeddings from ≥ 10 movie ratings improve the quality of the personalized MoE mapping.

Lesson 4 — Model Comparison

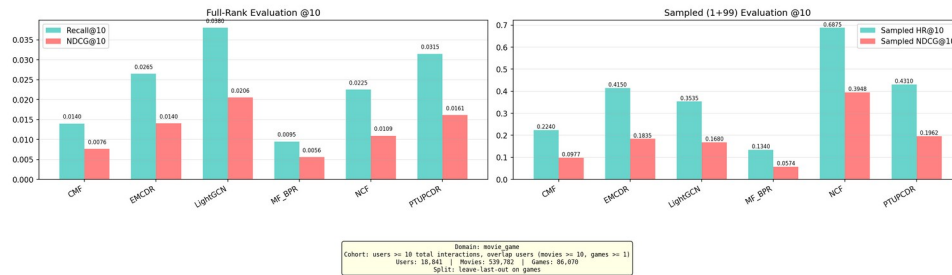


Figure 24: Lesson 4 — Source-rich overlap users

5.5 Lesson 5: Catalog Sharpening

Experiment Setup

Variable Changed: Movie catalog size — from 39,534 (L4) to 10,311 (L5) by removing movies with < 10 interactions, reducing the catalog by 74% while sharpening 75% of interactions.

Held Constant: Users (14,328), game catalog (9,147), model set, evaluation protocol, overlap (100%).

Expectation: Removing noisy long-tail movies should improve signal quality for collaborative models. SBERT-based models might suffer since they can leverage even rare items.

Key Finding

Removing 74% of the movie catalog has minimal impact. LightGCN drops -10%, PTUPCDR drops -12%, but NCF actually improves +54%. CDR models show mixed results, suggesting that catalog manipulation is not a reliable lever for improving CDR.

Lesson 5 — Model Comparison

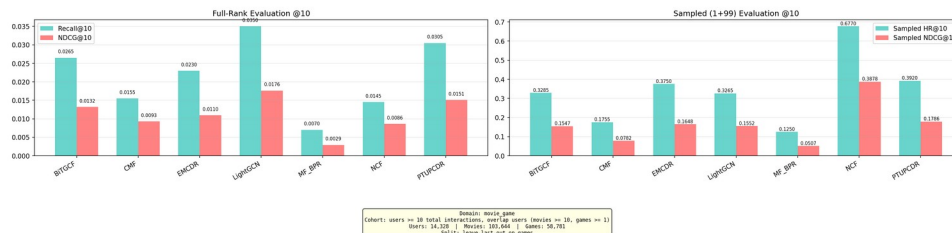


Figure 25: Lesson 5 — Catalog sharpening ablation

5.6 Lesson 6: Cold-Start Protocol

Experiment Setup

Variable Changed: Evaluation split — 80% warm users (full game history in training), 20% cold users (zero game interactions in training, only movie history). Evaluate on cold users only.

Held Constant: Data (overlap users), model set.

Expectation: CDR models should outperform single-domain models by 2–5× on cold users.

Model	Family	Recall@10	NDCG@10	vs LightGCN
Popularity	Baseline	0.0381	0.0194	5.7×
EMCDR	Mapping (CDR)	0.0334	0.0152	5.0×
PTUPCDR	Personal mapping (CDR)	0.0301	0.0124	4.5×
LightGCN	Graph (single)	0.0067	0.0034	1×
NCF	Neural (single)	0.0020	0.0008	0.3×
CMF	Joint MF (CDR)	0.0020	0.0015	0.3×
MF-BPR	MF (single)	0.0007	0.0002	0.1×

Key Finding

Confirmed — EMCDR achieves 5× over LightGCN on cold users. Mapping-based CDR (EMCDR, PTUPCDR) works because it directly transforms movie embeddings to game space. Popularity (0.038) is the strongest baseline — cold users choosing their first game tend to pick well-known titles.

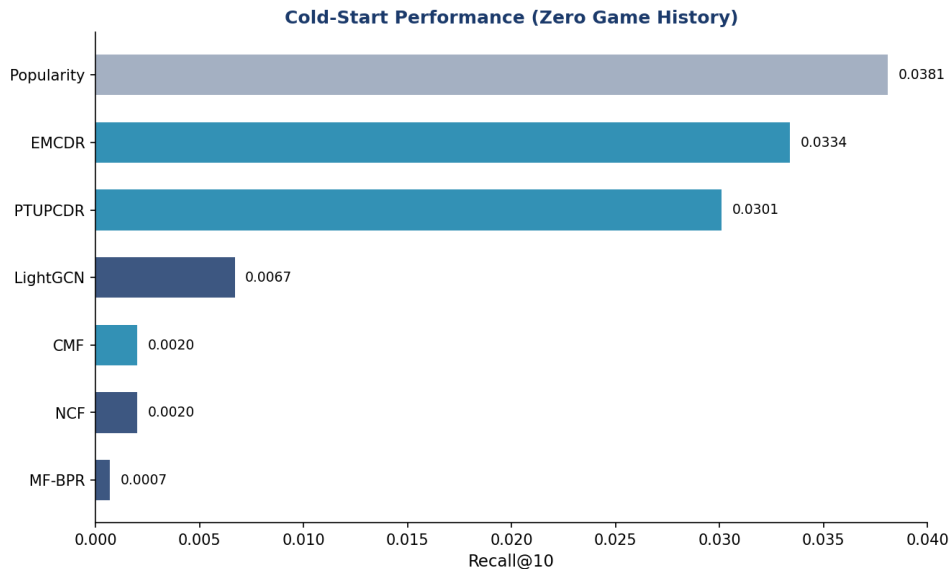


Figure 26: Cold-start Recall@10 — CDR mapping models (teal) vs single-domain (blue)

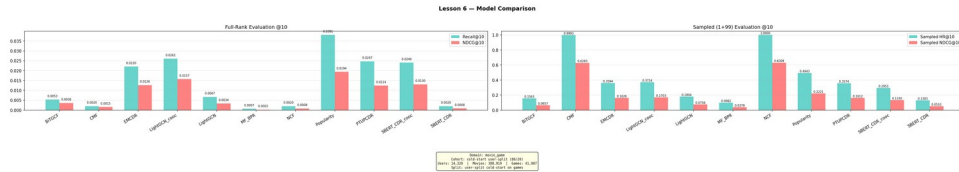


Figure 27: Lesson 6 — Cold-start: CDR vs single-domain on zero-game users

5.7 Lesson 7: Content-Aware CDR (SBERT)

Experiment Setup

Variable Changed: Model type — adding content-based SBERT and SBERT-CDR to the comparison.

Held Constant: Data (src-rich overlap), evaluation protocol.

Expectation: SBERT-CDR should bridge movie and game content in a shared semantic space, particularly for unpopular/niche target items.

Model	Overall Recall@10	Niche Subgroup Recall@10
LightGCN	0.0335	0.0122
SBERT-CDR	0.0330	0.1098 (9×)
SBERT (in-domain)	0.0310	0.1341 (11×)
PTUPCDR	0.0220	0.0000

Key Finding

SBERT-CDR nearly matches LightGCN overall (within 1.5%) and wins 9× on the 'one_shot_unpopular_target_user' subgroup — users with exactly 1 game training item whose test game is below median popularity. Text semantic similarity captures genre/theme alignment that collaborative methods cannot learn from sparse data.

Lesson 7 — Model Comparison

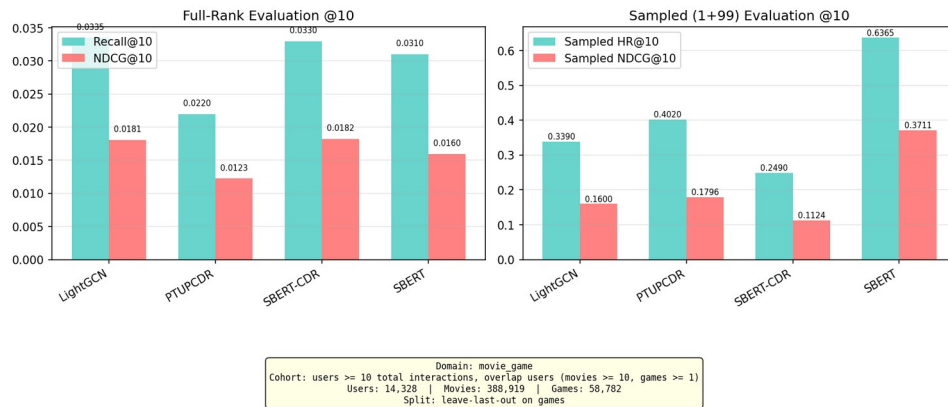


Figure 28: Lesson 7 — SBERT vs collaborative models

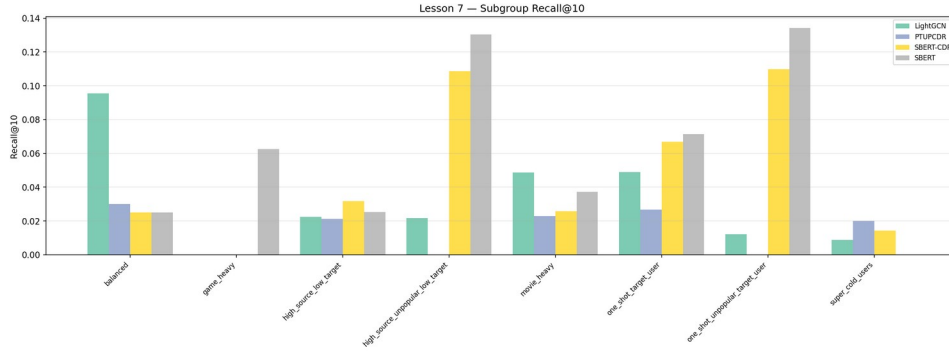


Figure 29: Lesson 7 — Subgroup analysis: SBERT dominates niche items

5.8 Lesson 8: Co-occurrence Reranking

We test whether a training-free movie→game co-occurrence reranking layer improves every model across all regimes.

LLO Results (Lesson 3 dataset, 100% overlap users)

Model	Base Recall@10	+Cooc Recall@10	Delta
LightGCN	0.0595	0.0625	+5%
MF-BPR	0.0445	0.0520	+17%
CMF	0.0395	0.0380	-4%
PTUPCDR	0.0315	0.0355	+13%
NCF	0.0255	0.0370	+45%
EMCDR	0.0225	0.0335	+49%

Cold-Start Results (Lesson 6 split)

Model	Base Recall@10	+Cooc Recall@10	Delta
Popularity	0.0381	0.0387	+1.6%
EMCDR	0.0334	0.0341	+2%
LightGCN	0.0067	0.0261	+289%
CMF	0.0007	0.0321	+45×

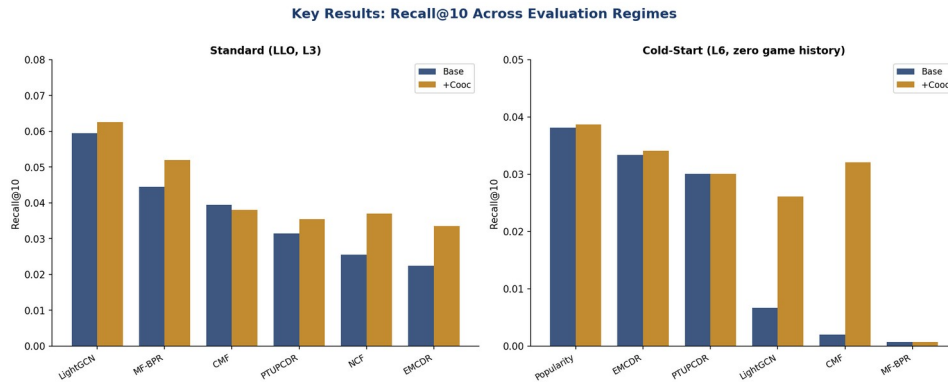


Figure 30: Key results summary — base vs cooc across LLO (left) and cold-start (right)

Key Finding

Cooc improves most models. The gain is inversely proportional to how well the base model already uses movie signal. Game-only models gain the most because movie history is entirely invisible to them. Properly-tuned CDR models (CMF) show slight negative delta because cooc adds redundant/conflicting signal.

Takeaway

Co-occurrence reranking should be applied universally as a post-processing layer. It is training-free, adds minimal latency, and provides consistent improvement across all model families. At cold-start, it rescues single-domain models by providing item-item signals when user-item signals are absent. The magnitude of improvement also serves as a diagnostic tool: if co-occurrence dramatically boosts a model's scores, the base model is likely misconfigured or undertrained.

5.9 Hyperparameter Analysis

During experimentation, we discovered that several models had significantly suboptimal default hyperparameters. Systematic sweeps revealed the following fixes:

Model	Parameter	Original	Fixed	Impact
MF-BPR	Learning rate	0.005	0.05	Loss stuck at 0.693 → converges. Recall 9×
CMF	Learning rate	0.01	0.0005	Too high for Adam → diverges
CMF	Alpha (domain balance)	0.3	0.05	Over-weighted 7× more movie interactions
NCF	Epochs	50	150	Under-trained. Modest improvement (~30%)
EMCDR	(all swept)	—	—	Architectural ceiling ~0.025. No fix possible

Takeaway

Optimizer choice dictates lr range. NumPy SGD (MF-BPR) needs lr=0.05 because there is no per-parameter adaptation. Adam-based models (CMF, NCF via RecBole) need lr=0.0005–0.001. Using SGD-appropriate lr with Adam causes divergence and vice versa.

6. Discussion

6.1 Routing Decision Rule

The experimental findings across all eight lessons converge on a practical routing rule: no single model dominates all user contexts. Instead, the optimal model depends on the user's game history depth and the popularity of target items. This routing rule is the primary practical output of this project.

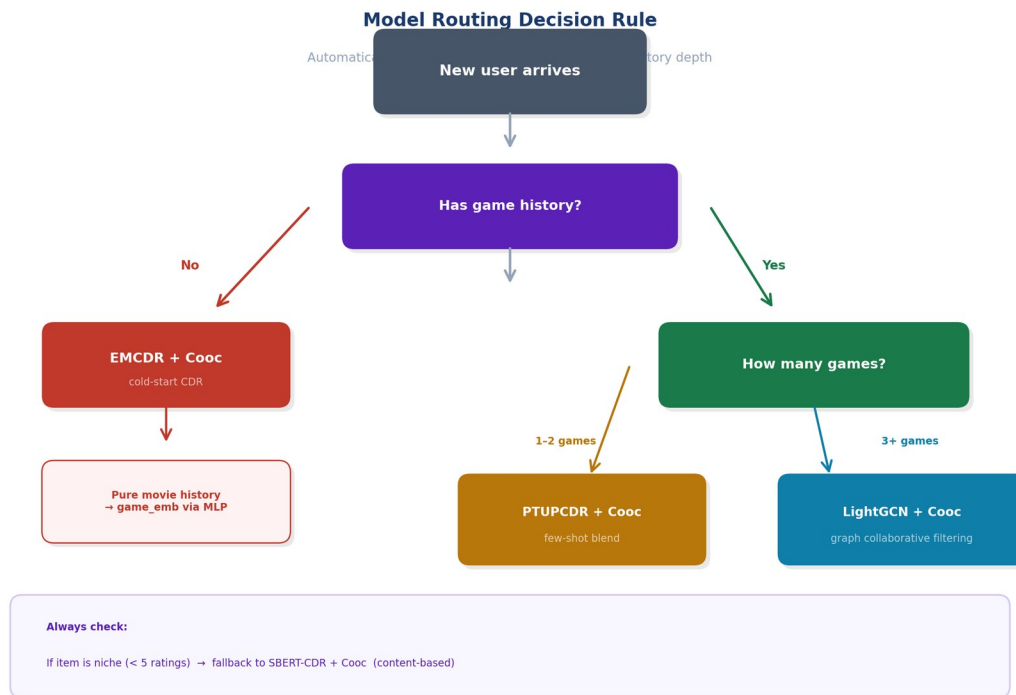


Figure 31: Model Routing Decision Rule

User State	Recommended Model	Justification
0 games, rich movies	EMCDR + cooc	L6: EMCDR best CDR at cold-start (global MLP works without game edges)
1–2 games, rich movies	PTUPCDR + cooc	L4: few-shot blend activates with 1+ game edge; L8: cooc +13%
3–9 games	LightGCN + cooc	L3: best LLO base model; L8: cooc +5%
10+ games	LightGCN + cooc	L2–L4: LightGCN dominates with sufficient game history

Niche / unpopular items	SBERT-CDR + cooc	L7: SBERT 9× on niche items; L8: cooc provides behavioral anchor
-------------------------	------------------	--

6.2 Key Insights

Overlap is the critical variable for CDR: The difference between L2 (5.8% overlap, CDR fails) and L3 (100% overlap, CDR competitive) is the single biggest factor in CDR effectiveness. Without overlap users, the mapping function has no training signal.

CDR's value is regime-specific: CDR adds the most value at cold-start (L6: 4× over LightGCN) and diminishes as game history grows. With 10+ game interactions, LightGCN's graph convolution captures user preference more effectively than cross-domain transfer.

Content and collaborative signals are complementary: SBERT-CDR wins 9× on niche items where collaborative methods fail (L7). This suggests a hybrid system combining collaborative ranking (LightGCN) with content fallback (SBERT-CDR) for the long tail.

Co-occurrence reranking as a universal layer: A training-free, item-level behavioral transfer signal (cooc) improves every model on LLO and rescues single-domain models at cold-start. Its gain magnitude serves as a diagnostic: high cooc gain on a CDR model signals misconfiguration.

Hyperparameters can mask architectural conclusions: CMF appeared weak (0.012) until we discovered lr/α were wrong (0.040 when fixed). EMCDR's cooc gain (+70%) appeared to validate the cooc layer but was actually masking a broken CMF baseline. Always tune before concluding.

7. System Design

Beyond the experimental benchmarks, we built a full-stack demonstration system that brings the research findings to life as an interactive web application. The system serves recommendations from all 9 models simultaneously, allowing users to compare collaborative, cross-domain, and content-based approaches side by side.

7.1 High-Level Architecture

The system follows a three-tier architecture: a Next.js 14 frontend, a FastAPI backend, and an offline ML pipeline. The backend loads pre-trained model embeddings into an in-memory store for sub-millisecond scoring, while a SQLite database persists user ratings across restarts. The ML pipeline processes raw Amazon Reviews data into Parquet files, trains all 8 models, and exports embeddings as NumPy arrays for the demo backend.



Figure 32: High-level system architecture — three-tier design with in-memory embedding store

The in-memory store holds 7 model embedding matrices (LightGCN games/movies, EMCDR, PTUPCDR, NCF, SBERT content embeddings, and co-occurrence matrices) totaling approximately 300MB. All scoring is performed as dot products between user and item embeddings, enabling real-time recommendation generation across all 9 rows.

7.2 Recommendation Flow

When a user requests recommendations, the HybridRecommender generates 9 parallel recommendation rows — 5 for games (purple lanes) and 4 for movies (blue lanes). Each row uses a different model or model combination, directly reflecting the experimental findings from Lessons 2–8.



Figure 33: Recommendation flow — 9 parallel rows using different model paradigms

Each row follows the same post-processing pipeline: score all items via embedding dot product, apply co-occurrence bonus (if applicable), mask to the correct domain, normalize scores, exclude already-rated items, take top-K, deduplicate across rows, and enrich with metadata. This ensures consistent behavior while allowing each row to demonstrate a different recommendation paradigm.

7.3 Frontend Architecture

The frontend is a Next.js 14 application with three main pages: a home page with user group selection (Balanced, Movie Heavy, Cold Start, Game Heavy), a recommendations page showing 9 model rows with horizontal card lanes, and an item detail page with star ratings and similar item sections. The domain visual language uses blue for movies and purple for games throughout.

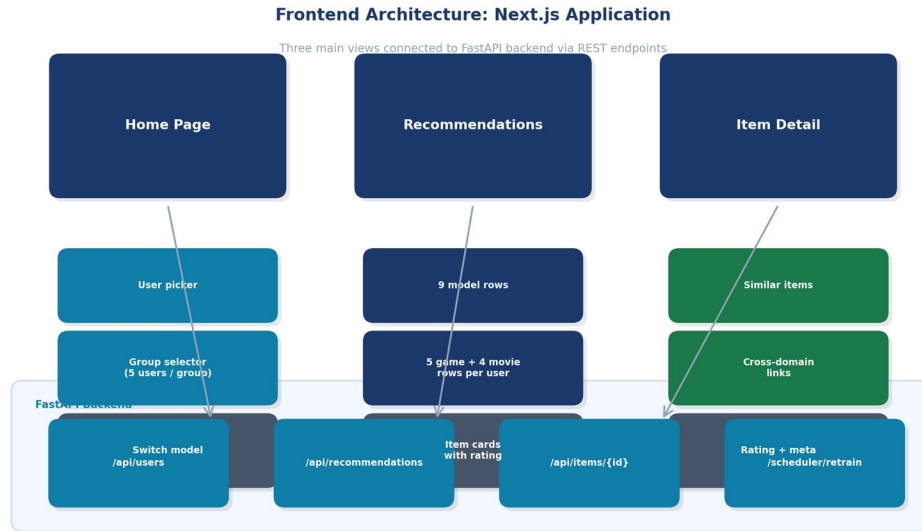


Figure 34: Frontend architecture — page structure and domain visual language

7.4 Refresh Architecture

The system implements three refresh tiers to balance latency and freshness. Tier 1 (Instant, ~1ms): Co-occurrence, SBERT, and popularity models require no retraining — a new rating immediately affects recommendations through dict lookups and profile recomputation. Tier 2 (Fast, ~50ms): The in-memory user ratings dict is updated on each API call, so score recomputation reflects the latest state. Tier 3 (Batch, ~40s): LightGCN requires full retraining, scheduled hourly via a background daemon thread with hot-swap of embeddings in the store.

7.5 Key Design Decisions

Decision	Rationale
Dual item space (SD + CD)	Single-domain models need compact game-only index; CDR models need unified movie+game index. Linked via Amazon ASIN.
In-memory embedding store	Dot product scoring is orders of magnitude faster in memory than any DB query. SQLite only for rating persistence.
Cooc as post-processing	Training-free, model-agnostic, instant refresh. Applied as score bonus to improve every model universally (Lesson 8).
Hourly LightGCN retrain	Full retrain takes ~40s. Real-time per-rating retraining is impractical. Cooc/SBERT fill the freshness gap.
9 rows (5 game + 4 movie)	Each row demonstrates a different paradigm: collaborative, CDR, content, behavioral, popularity — the full research spectrum.
User groups on home page	Different groups activate different strategies (routing rule

	from Lesson 6), making findings directly interactive.
--	---

8. Conclusion and Future Work

This project demonstrates that cross-domain recommendation from movies to games is effective under specific conditions: high user overlap ($\geq 80\%$), sufficient source domain history (≥ 10 ratings), and particularly at cold-start where users have no target-domain interactions. The optimal strategy is not a single model but a routing rule that matches the model to the user's context.

The co-occurrence reranking layer is the most practically impactful finding — it requires no training, operates at test time, and consistently improves every model. It also serves as a diagnostic tool for detecting CDR model misconfiguration.

Future Work

- Statistical significance: Run all experiments with 3–5 random seeds and report confidence intervals
- Additional domain pairs: Test the routing rule on other Amazon category pairs (e.g., books→movies, electronics→games)
- Online evaluation: Deploy the routing rule in an A/B test to measure real-world click-through improvements
- Hybrid model: Train a single model that combines LightGCN graph signal with SBERT content and cooc behavioral transfer
- Temporal dynamics: Investigate whether user preferences transfer differently over time (e.g., movie taste from 2020 may not predict 2024 game preferences)

9. References

- [1] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T. S. (2017). Neural collaborative filtering. WWW 2017.
- [2] He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). LightGCN: Simplifying and powering graph convolution network for recommendation. SIGIR 2020.
- [3] Singh, A. P., & Gordon, G. J. (2008). Relational learning via collective matrix factorization. KDD 2008.
- [4] Man, T., Shen, H., Jin, X., & Cheng, X. (2017). Cross-domain recommendation: An embedding and mapping approach. IJCAI 2017.
- [5] Zhu, Y., Tang, Z., Liu, Y., Zhuang, F., Xie, R., Zhang, X., Lin, L., & He, Q. (2022). Personalized transfer of user preferences for cross-domain recommendation. WSDM 2022.
- [6] Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian personalized ranking from implicit feedback. UAI 2009.
- [7] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. EMNLP 2019.
- [8] Hou, Y., Li, J., He, Z., Yan, A., Chen, X., & McAuley, J. (2024). Bridging language and items for retrieval and recommendation. arXiv preprint. (Amazon Reviews 2023 dataset)